RTOS UART
开发指南

版本号: 1.7
发布日期: 2025.04.29

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2020.7.7	AWA1636	初始版本
1.1	2021.4.14	AWA1636	增加 sys_config.fex
1.2	2023.2.15	XAA0249	修改格式
1.3	2023.12.18	XAA0312	修改测试 demo
1.4	2023.11.27	XAA0312	更新产品列表
1.5	2024.10.22	AWA1427	基于新模板更新文档
1.6	2025.04.07	XAA0249	增加 MR536 平台、新增接口与 FAQ 具体问题
1.7	2025.04.29	XAA0249	增加 rtos 切换时钟使用特殊波特率方法

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1
1.4 文档约定 2
1.4.1 标志说明 2
1.4.2 地址与数据描述方法约定 2
1.4.3 数值单位约定 2
1.5 相关术语介绍 3
1.5.1 硬件术语 3
1.5.2 软件术语 3

2 模块介绍 **4**
2.1 模块配置 4
2.1.1 menuconfig 配置说明 4
2.2 源码结构 4
2.3 软件框架 5

3 模块接口说明 **6**
3.1 hal_uart_get_version 6
3.2 hal_uart_init 7
3.3 hal_uart_init_for_amp_cli 7
3.4 hal_uart_deinit 7
3.5 hal_uart_send 7
3.6 hal_uart_receive 8
3.7 hal_uart_receive_no_block 8
3.8 hal_uart_control 8
3.9 hal_uart_receive_polling 9
3.10 hal_uart_check_poll_state 9
3.11 hal_uart_poll_wakeup 9
3.12 hal_uart_register_poll_wakeup 9
3.13 hal_uart_set_hardware_flowcontrol 10
3.14 hal_uart_disable_flowcontrol 10
3.15 hal_uart_set_loopback 10
3.16 hal_uart_enable_rx 10
3.17 hal_uart_disable_rx 11
3.18 hal_callback_install 11
3.19 hal_uart_set_mailbox_size 11

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 功能开发 **12**
4.1 功能概述 12
4.2 开发流程 12
4.2.1 初始化某个 uart port 12
4.2.2 反初始化某个 uart port 12
4.2.3 发送接收数据 12
4.3 编程示例 13

5 调试方法 **15**
5.1 调试工具 15
5.1.1 hal_uart 15
5.1.2 hal_uart_stress 15
5.1.3 hal_uart_dma 16

6 FAQ **17**
6.1 如何修改 DEBUG 串口的默认波特率 17
6.2 一号通 FAQ 列表 17
6.3 关于 uart 波特率 17
6.3.1 波特率计算公式与大致原理 17
6.3.2 如何计算当前配置下的误码率 19
6.3.3 如何设置当前平台支持某个特定波特率 19
6.3.3.1 判断默认配置是否满足波特率要求 20
6.3.3.2 修改 uart 平台文件中时钟配置 20
6.3.4 如何设置不同的波特率 21

版权所有 © 珠海全志科技股份有限公司。保留一切权利

表　格

表 1-1　适用产品列表 1
表 1-2　地址与数据描述方法约定 2
表 1-3　数值单位约定 2
表 1-4　硬件术语 3
表 1-5　软件术语 3
表 3-1　模块接口列表 6

广州星翼电子科技有限公司zdyzlinux

1 前言

1.1 文档简介

本文档介绍 RTOS 平台 UART 驱动的接口及使用方法。

1.2 目标读者

- RTOS UART 驱动开发工程师
- RTOS UART 驱动使用工程师

1.3 适用范围

表 1-1: 适用产品列表

产品名称	内核版本	驱动文件
AI985	FreeRTOS	hal_uart.c
F133	Melis	hal_uart.c
MR153	FreeRTOS	hal_uart.c
MR527	FreeRTOS	hal_uart.c
R328	FreeRTOS	hal_uart.c
R329-DSP	FreeRTOS	hal_uart.c
R128	FreeRTOS	hal_uart.c
V833	Melis	hal_uart.c
V821	FreeRTOS	hal_uart.c
XR875	FreeRTOS	hal_uart.c

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.4 文档约定

1.4.1 标志说明

注意

- **提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。**

说明

为准确理解文中指令、正确实施操作而提供的补充或强调信息。

技巧

一些容易忽视的小功能、技巧。了解这些功能或技巧能帮助解决特定问题或者节省操作时间。

1.4.2 地址与数据描述方法约定

本文档在描述地址、数据时遵循如下约定：

表 1-2: 地址与数据描述方法约定

符号	例子	说明
0x	0x0200，0x79	地址或数据以 16 进制表示。
0b	0b010，0b00 000 111	数据采用二进制表示 (寄存器描述除外)。
X	00X，XX1	数据描述中，X 代表 0 或 1。 例如，00X 代表 000 或 001；XX1 代表 001，011，101 或 111。

1.4.3 数值单位约定

本文档在描述数据容量（如 NAND 容量）时，单位词头代表的是 1024 的倍数；描述频率、数据速率等时则代表的是 1000 的倍数。具体如下：

表 1-3: 数值单位约定

类型	符号	对应数值
数据容量（如 NAND 容量）	1 K	1024
	1 M	1 048 576
	1 G	1 073 741 824
频率，数据速率等	1 k	1000

版权所有 © 珠海全志科技股份有限公司。保留一切权利

类型	符号	对应数值
	1 M	1 000 000
	1 G	1 000 000 000

1.5 相关术语介绍

1.5.1 硬件术语

表 1-4: 硬件术语

术语	解释说明
UART	Universal Asynchronous Receiver/Transmitter

1.5.2 软件术语

表 1-5: 软件术语

术语	解释说明
HAL	Hardware Abstraction Layer，硬件抽象层
BaudRate	波特率，UART 通信中每秒钟传输符号的数量

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 模块介绍

2.1 模块配置

2.1.1 menuconfig 配置说明

```
Drivers Options  --->
  soc related device drivers  --->
      [*] enable uart driver
      [ ]  enable uart hal APIs test command
      [ ]  support poll APIs
      [ ]  support uart0 device
      [ ]  support uart1 device
      [ ]  support uart2 device
      ....
      (2)  cli uart port number
      [ ]  support transfer uart data through dma
```

说明

cli uart port number 填入的序号是串口控制台使用的 UART 口

2.2 源码结构

1. 驱动源码

```
hal/source/uart/
├── hal_uart.c
├── Kconfig
├── Makefile
├── platform # 平台相关的寄存器配置信息
│   ├── uart-sun20iw2p1.h
│   ├── uart-sun300iw1.h
│   ├── uart-sun50iw11.h
│   ├── uart-sun55iw3.h
│   ├── uart-sun55iw6.h
│   ├── uart-sun60iw1.h
│   ├── uart-sun8iw18.h
│   ├── uart-sun8iw19.h
│   ├── uart-sun8iw20.h
│   └── uart-sun8iw21.h
|   ...
├── platform-uart.h
└── uart.h
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2. 驱动 APIs 声明头文件

```
include/hal/
└──── hal_uart.h
```

3. 驱动 APIs 测试代码

```
hal/test/uart/
├──── Makefile
└──── test_uart.c
```

2.3 软件框架

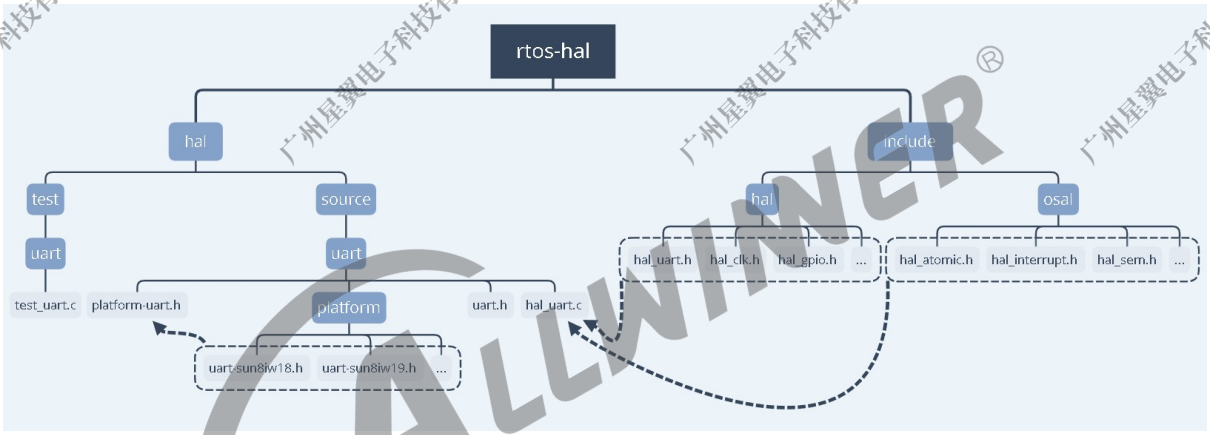

图 2-1: RTOS UART 驱动框架

广州星翼电子科技有限公司zdyzlinux

3 模块接口说明

表 3-1: 模块接口列表

API	解释说明
hal_uart_get_version	获取 uart 驱动版本号
hal_uart_init	初始化 uart 驱动
hal_uart_init_for_amp_cli	作为控制台初始化 uart 驱动
hal_uart_deinit	卸载 uart 驱动
hal_uart_send	发送数据
hal_uart_receive	接收数据
hal_uart_receive_no_block	非阻塞接收
hal_uart_control	设置 uart 工作参数
hal_uart_receive_polling	轮询接收
hal_uart_check_poll_state	查看轮询接收状态
hal_uart_poll_wakeup	唤醒接收
hal_uart_set_hardware_flowcontrol	设置硬件流控
hal_uart_disable_flowcontrol	禁止硬件流控
hal_uart_check_poll_state	查看轮询接收状态
hal_uart_set_loopback	设置回环模式
hal_uart_enable_rx	使能接收
hal_uart_disable_rx	关闭接收
hal_callback_install	注册中断回调函数 (部分平台支持)
hal_uart_set_mailbox_size	设置 mailbox 大小 (部分平台支持)

3.1 hal_uart_get_version

- 作用：获取 UART 驱动的版本号
- 参数：
  - dev:UART 端口号
- 返回：
  - 非空:UART 驱动版本号

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.2 hal_uart_init

- 作用：初始化 UART 驱动
- 参数：
  - uart_port:UART 端口号
- 返回：
  - SUNXI_HAL_OK: 成功
  - HAL_UART_STATUS_ERROR: 失败

3.3 hal_uart_init_for_amp_cli

- 作用：初始化 UART 驱动作为控制台
- 参数：
  - uart_port:UART 端口号
- 返回：
  - SUNXI_HAL_OK: 成功
  - HAL_UART_STATUS_ERROR: 失败

3.4 hal_uart_deinit

- 作用：卸载 UART 驱动
- 参数：
  - uart_port:UART 端口号
- 返回：
  - SUNXI_HAL_OK: 成功

3.5 hal_uart_send

- 作用：发送数据
- 参数：
  - dev:UART 端口号
  - data: 发送数据缓冲区
  - num: 发送数据长度

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 返回：
  - size: 成功发送的字节数

3.6 hal_uart_receive

- 作用：接收数据
- 参数：
  - dev:UART 端口号
  - data: 接收数据缓冲区
  - num: 接收数据长度
- 返回：
  - size: 成功接收的字节数

3.7 hal_uart_receive_no_block

- 作用：非阻塞接收
- 参数：
  - dev:UART 端口号
  - data: 接收数据缓冲区
  - num: 接收数据长度
  - timeout: 等待超时时长
- 返回：
  - size: 成功接收的字节数

3.8 hal_uart_control

- 作用：设置 uart 工作参数
- 参数：
  - uart_port:UART 端口号
  - cmd: 保留
  - args: 指向 _uart_config_t 类型的结构体
- 返回：
  - SUNXI_HAL_OK: 成功设置

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.9 hal_uart_receive_polling

- 作用：轮询接收
- 参数：
  - dev:UART 端口号
  - data: 接收数据缓冲区
  - num: 接收数据长度
- 返回：
  - size: 成功接收的字节数

3.10 hal_uart_check_poll_state

- 作用：查看轮询接收状态
- 参数：
  - dev_id:UART 端口号
  - key: 标志位，取值 POLLIN、POLLOUT、POLLERR
- 返回：
  - mask:uart 当前接收状态

3.11 hal_uart_poll_wakeup

- 作用：执行唤醒处理函数
- 参数：
  - dev_id:UART 端口号
  - key: 标志位，取值 POLLIN、POLLOUT、POLLERR
- 返回：
  - mask: 回调函数执行返回值

3.12 hal_uart_register_poll_wakeup

- 作用：注册唤醒处理函数
- 参数：
  - poll_wakeup: 回调函数

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 返回：
  - 0

3.13 hal_uart_set_hardware_flowcontrol

- 作用：设置硬件流控
- 参数：
  - uart_port:UART 端口号
- 返回：
  - void

3.14 hal_uart_disable_flowcontrol

- 作用：禁止硬件流控
- 参数：
  - uart_port:UART 端口号
- 返回：
  - void

3.15 hal_uart_set_loopback

- 作用：设置 uart 回环
- 参数：
  - uart_port:UART 端口号
  - enable: 是否开启
- 返回：
  - void

3.16 hal_uart_enable_rx

- 作用：使能 rx 中断
- 参数：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

  - uart_port:UART 端口号
- 返回：
  - 0

3.17 hal_uart_disable_rx

- 作用：关闭 rx 中断
- 参数：
  - uart_port:UART 端口号
- 返回：
  - 0

3.18 hal_callback_install

- 作用：注册中断回调函数 (部分平台支持)
- 参数：
  - uart_port:UART 端口号
  - callback: 回调函数
- 返回：
  - HAL_UART_STATUS_OK: 成功
  - HAL_UART_STATUS_ERROR: 失败

3.19 hal_uart_set_mailbox_size

- 作用：设置 mailbox 大小 (部分平台支持)
- 参数：
  - uart_port:UART 端口号
  - size: 设置 mailbox 大小
- 返回：
  - void

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 功能开发

4.1 功能概述

UART 驱动提供了以下核心功能：

- UART 硬件初始化;
- 数据发送和接收;
- 波特率设置;
- 数据位、停止位和校验位配置;
- 发送、接收缓冲区管理;

4.2 开发流程

4.2.1 初始化某个 uart port

步骤 1 定义 _uart_config_t 类型 uart 参数变量，并配置波特率、停止位等信息。

步骤 2 调用 hal_uart_init 初始化指定 uart port。

步骤 3 调用 hal_uart_control 设置 uart 参数。

步骤 4 调用 hal_uart_set_hardware_flowcontrol 或 hal_uart_disable_flowcontrol 设置流控。

4.2.2 反初始化某个 uart port

步骤 1 调用 hal_uart_deinit 反初始化

4.2.3 发送接收数据

步骤 1 调用 hal_uart_send 发送数据。

步骤 2 调用 hal_uart_receive 或 hal_uart_receive_no_block 接收数据。

说明

默认未使用 DMA，若需要请打开 CONFIG_DRIVERS_UART 配置。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.3 编程示例

参考代码：rtos-hal/hal/test/uart/test_uart.c

```
int cmd_test_uart(int argc, char **argv)
{
    uint8_t tbuf[6] = {"hello"};
    uint8_t rbuf[10] = {0};
    uart_port_t port;
    uint32_t baudrate;
    _uart_config_t uart_config;
    int i;

    hal_log_info("Testing UART in loopback mode");

    if (argc != 3) {
        cmd_usage();
        return -1;
    }

    port = strtol(argv[1], NULL, 0);
    baudrate = strtol(argv[2], NULL, 0);

    if(CONFIG_CLI_UART_PORT == port){
        hal_log_info("uart0 can't test, please use other port!");
        return -1;
    }
    memset(rbuf, 0, 10 * sizeof(uint8_t));
    switch (baudrate) {
        case 4800:
            uart_config.baudrate = UART_BAUDRATE_4800;
            break;

        case 9600:
            uart_config.baudrate = UART_BAUDRATE_9600;
            break;

        case 38400:
            uart_config.baudrate = UART_BAUDRATE_38400;
            break;

        case 115200:
            uart_config.baudrate = UART_BAUDRATE_115200;
            break;

        case 230400:
            uart_config.baudrate = UART_BAUDRATE_230400;
            break;

        case 460800:
            uart_config.baudrate = UART_BAUDRATE_460800;
            break;

        case 921600:
            uart_config.baudrate = UART_BAUDRATE_921600;
            break;

        case 1500000:
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
            uart_config.baudrate = UART_BAUDRATE_1500000;
            break;

        case 4000000:
            uart_config.baudrate = UART_BAUDRATE_4000000;
            break;
        default:
            hal_log_info("Using default baudrate: 115200");
            uart_config.baudrate = UART_BAUDRATE_115200;
            break;
    }

    uart_config.word_length = UART_WORD_LENGTH_8;
    uart_config.stop_bit = UART_STOP_BIT_1;
    uart_config.parity = UART_PARITY_NONE;

    /* 初始化uart */
    hal_uart_init(port);
    /* 设置各项配置（波特率、数据位、停止位等）*/
    hal_uart_control(port, 0, &uart_config);
    /* 关闭流控 */
    hal_uart_disable_flowcontrol(port);
    /* 设置软件回环，若硬件回环省略此步骤 */
    hal_uart_set_loopback(port, 1);

    /* send */
    hal_uart_send(port, tbuf, 5);

    /* loopback receive */
    hal_uart_receive_no_block(port, rbuf, 5, MS_TO_OSTICK(1000));

    printf("Sending:");
    for (i = 0; i < 5; i++)
        printf("%c", tbuf[i]);
    printf("\n");

    printf("Receiving:");
    for (i = 0; i < 5; i++)
        printf("%c", rbuf[i]);
    printf("\n");

    ....
    ....

    /* 反初始化uart */
    hal_uart_deinit(port);

    return 0;
}
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 调试方法

5.1 调试工具

配置 CONFIG_HAL_TEST_UART，可使用一些测试命令

```
Drivers Options ---->
  soc related device drivers  ---->
    [*] nable uart hal APIs test command
```

5.1.1 hal_uart

- 使用方法：hal_uart port baudrate
- 用途：测试 uart 的基础功能，包含自发自收。

使用示例：

```
cpu0>hal_uart 2 115200
Testing UART in loopback mode
Sending:hello
Receiving:hello
Test hal_uart_init API success!
Test hal_uart_control API success!
Test hal_uart_disable_flowcontrol API success!
Test hal_uart_set_loopback API success!
Test hal_uart_send API success!
Test hal_uart_receive API success!
Test hal_uart_deinit API success!
Test uart hal APIs success!
```

5.1.2 hal_uart_stress

- 使用方法：hal_uart_stress port baudrate flowctrl loopback len
- 用途：测试 UART 的数据收发

使用示例：

```
uart>hal_uart_stress 2 115200 0 1 128
Testing UART in loopback mode with stress
[E(pm)] syscore: sunxi_uart(81129488) has already registered
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
flow:0, loopback:1 len:128
send
send done
recv
recv done
Sending:
0x61 0x62 0x63 0x64 0x65 0x66 0x67 0x68 0x69 0x6a 0x6b 0x6c 0x6d 0x6e 0x6f 0x70
0x71 0x72 0x73 0x74 0x75 0x76 0x77 0x78 0x79 0x7a 0x7b 0x7c 0x7d 0x7e 0x7f 0x80
0x81 0x82 0x83 0x84 0x85 0x86 0x87 0x88 0x89 0x8a 0x8b 0x8c 0x8d 0x8e 0x8f 0x90
0x91 0x92 0x93 0x94 0x95 0x96 0x97 0x98 0x99 0x9a 0x9b 0x9c 0x9d 0x9e 0x9f 0xa0
0xa1 0xa2 0xa3 0xa4 0xa5 0xa6 0xa7 0xa8 0xa9 0xaa 0xab 0xac 0xad 0xae 0xaf 0xb0
0xb1 0xb2 0xb3 0xb4 0xb5 0xb6 0xb7 0xb8 0xb9 0xba 0xbb 0xbc 0xbd 0xbe 0xbf 0xc0
0xc1 0xc2 0xc3 0xc4 0xc5 0xc6 0xc7 0xc8 0xc9 0xca 0xcb 0xcc 0xcd 0xce 0xcf 0xd0
0xd1 0xd2 0xd3 0xd4 0xd5 0xd6 0xd7 0xd8 0xd9 0xda 0xdb 0xdc 0xdd 0xde 0xdf 0xe0
Receiving:
0x61 0x62 0x63 0x64 0x65 0x66 0x67 0x68 0x69 0x6a 0x6b 0x6c 0x6d 0x6e 0x6f 0x70
0x71 0x72 0x73 0x74 0x75 0x76 0x77 0x78 0x79 0x7a 0x7b 0x7c 0x7d 0x7e 0x7f 0x80
0x81 0x82 0x83 0x84 0x85 0x86 0x87 0x88 0x89 0x8a 0x8b 0x8c 0x8d 0x8e 0x8f 0x90
0x91 0x92 0x93 0x94 0x95 0x96 0x97 0x98 0x99 0x9a 0x9b 0x9c 0x9d 0x9e 0x9f 0xa0
0xa1 0xa2 0xa3 0xa4 0xa5 0xa6 0xa7 0xa8 0xa9 0xaa 0xab 0xac 0xad 0xae 0xaf 0xb0
0xb1 0xb2 0xb3 0xb4 0xb5 0xb6 0xb7 0xb8 0xb9 0xba 0xbb 0xbc 0xbd 0xbe 0xbf 0xc0
0xc1 0xc2 0xc3 0xc4 0xc5 0xc6 0xc7 0xc8 0xc9 0xca 0xcb 0xcc 0xcd 0xce 0xcf 0xd0
0xd1 0xd2 0xd3 0xd4 0xd5 0xd6 0xd7 0xd8 0xd9 0xda 0xdb 0xdc 0xdd 0xde 0xdf 0xe0
```

5.1.3 hal_uart_dma

- 使用方法：hal_uart_dma port baudrate date_len num
- 用途：测试 UART DMA 数据收发

使用示例：

```
hal_uart_dma 2 115200 128 1
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 FAQ

6.1 如何修改 DEBUG 串口的默认波特率

路径：lichee/rtos-hal/hal/source/uart/hal_uart.c

```
/* default uart config */
_uart_config_t uart_defconfig =
{
    .baudrate    = UART_BAUDRATE_115200,
    .word_length = UART_WORD_LENGTH_8,
    .stop_bit    = UART_STOP_BIT_1,
    .parity      = UART_PARITY_NONE,
};
```

修改.baudrate 值即可，如修改为 UART_BAUDRATE_921600

6.2 一号通 FAQ 列表

- FAQ92 如何设置 1.5M 速率打印
- FAQ2436 melis 访问 B 盘设备节点访问部分 UART 设备失败问题解决
- FAQ2590 DSP 的串口功能在关闭 UART 引脚后仍能使用问题解决

6.3 关于 uart 波特率

6.3.1 波特率计算公式与大致原理

uart 波特率计算公式为：

apb clk rate = apb parent clk / apb div

baudrate = apb clk rate/ uart divisor / 16

如果已经理解以上公式可以直接跳转查看 “如何设置当前平台支持某个特定波特率” 章节，如果不理解请先看完本章节，了解基本原理

在不同的 Sunxi 硬件平台中，UART 控制器的时钟源选择、配置略有不同，总体上的时钟关系如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
hosc--------|
clk32K------|
clk16M_RC---|->apb mux->/apb div->apb_clk->uart_clk->/uart divisor-->/16->baud
pll_periX1--|
pll_periX2--|
...---------|
```

uart 波特率依赖 uart 时钟及其上层时钟，整体分为两个部分

- apb 部分
- apb 只是一个代称，在不同平台中名称可以是 apb1、apb_uart、apbs1 等，可以在时钟代码中查看

“‘ 以 1922 平台为例，ccu 文件路径为 SDK/rtos/lichee/rtos-hal/hal/source/ccu/platform/sun8iw22/sun8iw22_sys_ccu.c 在文件中搜索 uart，跳转至任意一个 uart 的时钟描述，可以从下列代码中看出此平台 uart 时钟的时钟源为 CLK_APB_UART

AW_CCU_GATE(uart1_bus_clk, “uart1-bus”, 0, MAKE_CLKn(AW_SYS_CCU, CLK_APB_UART), 0xE04, 0);

以上查询方法也可换成在 ccmu spec 中搜索 uart 时钟，并根据 spec 描述确定所用时钟源 “‘

- apb 时钟多有种时钟源可以选择，发布 SDK 时，一般选择 hosc 做时钟源，后期用户可在合理范围内自由调整

“‘ 以 1922 平台为例，平台文件为 SDK/rtos/lichee/rtos-hal/hal/source/uart/platform/uart-sun8iw21.h 在同一个文件中继续搜索 “apb-uart”，跳转至时钟定义处，可以看到 apb_uart_clk 可选择的所有时钟源有 5 个

const clk_number_t apb_uart_parents[] = { MAKE_CLKn(AW_SRC_CCU, CLK_SRC_HOSC), MAKE_CLKn(AW_SRC_CCU, CLK_SRC_LOSC), MAKE_CLKn(AW_SRC_CCU, CLK_SRC_RC_HF), MAKE_CLKn(AW_SYS_CCU, CLK_PLL_PERI0_600M), MAKE_CLKn(AW_SYS_CCU, CLK_PLL_PERI0_480M), };

AW_CCU_MUX(apb_uart_clk, “apb-uart”, 0, apb_uart_parents, 0x538, 24, 3);

当想要使用 “pll-peri0-480m” 时钟作为 apb_uart_clk 的父时钟时（1）在代码中搜索 “pll-peri0-480m”，跳转至时钟定义处 AW_CCU_LDIV(pll_peri0_480m_clk, “pll-peri0-480m”, CLK_IS_CRITICAL, MAKE_CLKn(AW_SYS_CCU, CLK_PLL_PERI0), 0x00A0, 2, 3);

（2）根据时钟定义处的时钟变量名称 “pll_peri0_480m_clk” 继续跳转，找到宏定义名称，将 CLK_PLL_PERI0_480M 填入平台文件的 SCLK #define UART_PCLK CLK_PLL_PERI0_480M

以上查询方法也可换成在 ccmu spec 中搜索 apb_uart 时钟，并根据 spec 描述确定可选时钟源及 div 范围 “‘

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- apb 时钟有 div 可对时钟源进行除频，可以将较大的频点整除成较小的频点，一般范围为 1-32，具体除频范围需要查阅 ccmu spec
- uart 部分
- uart clk 直接继承 apb 时钟频点，可以理解为 uart clk rate = apb clk rate
- uart 驱动内部有 uart divisor div 可以对 uart clk 再进行分频，此参数在 uart 内部自动设置，无法也无需手动调整，分频范围是 0-65535

6.3.2 如何计算当前配置下的误码率

需要知道以下信息：

- apbrate（apb 时钟频点）
- baudrate（期望波特率）
- 期望分频系数 uart divisor = apbrate / 16 / baudrate（可能是小数）
- 实际分频数 actual divisor = 四舍五入 uart divisor

误码率 = （uart divisor - actual divisor）/ uart divisor

连续传输数据量越大，对误码率要求越高，推荐不超过 1.725%，如果超过，可以考虑使用其他大小接近但误码率更小的波特率

```
示例1：
apb=24M
baudrate=115200
uart divisor = apbrate / 16 / baudrate = 24000000/16/115200=13.0208
actual divisor =13
误码率 = （13.0208 - 13）/ 13.0208 = 0.16%

示例2：
apb=160M
baudrate=10M
uart divisor = apbrate / 16 / baudrate = 160M/16/10M=1
actual divisor =1
误码率 = （1 - 1）/ 1 = 0%
```

6.3.3 如何设置当前平台支持某个特定波特率

上一章节已经描述了波特率的计算公式，并讲解了其计算原理，本章节将描述如何判断是否需要修改平台文件以及怎样修改

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.3.3.1 判断默认配置是否满足波特率要求

考虑到功耗，UART 平台文件中默认使用 hosc 做时钟源，一般是 24M

当不设置 apb 分频以及 uart divisor 时，baudrate = 24MHz/1/1/16=1.5MHz，即此时最大仅可支持 1.5M 的波特率

注意：最大支持 1.5M 不等于 1.5M 以下的波特率都可以支持！频点与波特率是离散的对应关系！

此时有两种情况需要修改 dtsi:

- 想要使用的波特率大于 1.5M
- 虽然想要使用的波特率小于 1.5M，但当前配置计算出来的波特率与目标波特率相差过大，误码率过大

6.3.3.2 修改 uart 平台文件中时钟配置

```
本处以1921平台为例
#define RESET_TYPE     AW_APP_CCU
#define CLK_TYPE       AW_APP_CCU

+ #define UART_SCLK          CLK_APP_APB1
+ #define SCLK_TYPE          AW_APP_CCU
+ #define UART_PCLK          CLK_APP_PERI_PLL_1X_FIXED_DIV
+ #define PCLK_TYPE          AW_APP_CCU
+ #define UART_SCLK_RATE     100000000

#define SUNXI_CLK_UART0 CLK_APP_BUS_UART0
#define SUNXI_RST_UART0 RST_APP_BUS_UART0
```

以上述代码修改为例讲解平台文件中有关时钟的两个重要节点

- SCLK
- SCLK 为当前 uart 的父时钟，用于设置例如 24M 等时钟频点
- PCLK
- PCLK 为 SCLK 的父时钟，用来规定 SCLK 频点，例如 480M 的 PCLK 可以分出 160M 的 SCLK
- CLK_TYPE
- 在 ccu 相关代码中可以看到如下定义

```
const clk_number_t apb_uart_parents[] =
{
    MAKE_CLKn(AW_SRC_CCU, CLK_SRC_HOSC),
    MAKE_CLKn(AW_SRC_CCU, CLK_SRC_LOSC),
    MAKE_CLKn(AW_SRC_CCU, CLK_SRC_RC_HF),
    MAKE_CLKn(AW_SYS_CCU, CLK_PLL_PERI0_600M),
    MAKE_CLKn(AW_SYS_CCU, CLK_PLL_PERI0_480M),
};
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

其中对于始终的描述第一个参数例如 AW_SRC_CCU 为 CLK_TYPE，第二个参数 CLK_SRC_LOSC 例如为时钟名

- 重要注意事项！！！
- 修改 APB 总线时钟，可能会影响到使用到 APB 总线时钟的相应模块，例如 twi 等，需要根据 ccmu spec 查询是否会影响到其他模块

示例 1：

```
例如需要配置uart2的波特率为460800HZ，先按照波特率公式倒推符合要求的时钟频点
baudrate = apb clk rate/ uart divisor / 16可以转换成apb clk rate = baudrate * uart divisor * 16，可以先假设uart divisor=1
        ，最小的符合要求的apb clk rate
apb clk rate = 460800 * 1 * 16 = 7372800
只要apb时钟是7372800的整数倍就可以支持到此频点，由于7372800无法直接用常用的480M、600M等整除，只能将时钟设
        置到接近的频点

例如：
50M接近7372800*7=51609600，在apb时钟可分出50M的情况下（如600M/12），就可以选取50M 作为apb的时钟频点，此时
        真实波特率为50M/7/16=446428
30M接近7372800*4=29491200，在apb时钟可分出30M的情况下（如480M/16），也可以选取30M作为apb的时钟频点，此时
        真实波特率为30M/4/16=468750

可以看出不同时钟选择造成误码率不同，符合外设要求即可使用
```

示例 2：

```
例如需要配置uart2的波特率为10M，先按照波特率公式倒推符合要求的时钟频点
baudrate = apb clk rate/ uart divisor / 16可以转换成apb clk rate = baudrate * uart divisor * 16，可以先假设uart divisor=1
        ，最小的符合要求的apb clk rate
apb clk rate = 10000000 * 1 * 16 = 16000000
只要apb时钟是16000000的整数倍就可以支持到此频点，由于160000000可以直接用常用的480M整除，可以将时钟设置到精
        确的频点
此时需要先确定平台是否支持将apb parent选到pll_peri_480M，若支持，可将apb时钟频点设置到160M，波特率为160M
        /1/16=10M
若不支持，则设计上无法支持到10M波特率
```

6.3.4 如何设置不同的波特率

注意 “设置波特率” 和“设置支持某个波特率的时钟”是不同的概念

- 设置波特率

指的是在用户使用 uart 时，可以通过uart_set_baudrate等接口设置某笔传输的波特率，是可以灵活配置的，每一次传输都可以重新设置

- 设置支持某个波特率的时钟

指的是在用户有想要使用的特定波特率的情况下，发现当前时钟无法支持到这个特定的波特率，就需要提前在平台文件中设置 uart 父时钟及频点、父时钟的父时钟，设置后需要重新编译，生成固件后就不可再更改

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。